Hourly Out-of-Stock Prediction for Fresh Food Retail

Ha Thuc Khuong Ninh[1]

FPT University, Ho Chi Minh City, Vietnam
hathuckhuongninh@gmail.com

Abstract. Accurate short-term stockout forecasting is important for fresh food retail because missed stockouts reduce service quality and unnecessary alerts increase replenishment effort. This report consolidates one data-processing and exploratory analysis workflow and five model experiments: XGBoost, LightGBM, Random Forest, Logistic Regression, and MLP. All models are evaluated on the FreshRetailNet-50K City ID 3 subset with 238,320 records, 101 stores, 173 products, and 90 days. The task predicts 16 hourly stockout labels for the 6:00–22:00 period (hourly slots 6–21) using 15 input features, a chronological 83-day/7-day split, hour-specific Precision–Recall thresholds, macro-averaged metrics, feature importance, and SHAP checks. XGBoost achieves the best Accuracy (80.20%), Recall (69.72%), and F1-Score (61.36%), while LightGBM achieves the highest Precision (55.70%) with a similar F1-Score (61.34%).

Keywords: Fresh food retail · Stockout forecasting · Multi-output classification · Hour-specific thresholding · Explainable AI · FreshRetailNet-50K.

1 Introduction

Inventory management in fresh food retail is difficult because products have short shelf lives, hourly demand changes quickly, and replenishment decisions must be made before shelves become empty. In this context, a zero-sale observation is ambiguous: it can indicate either no demand or unavailable stock. Predicting stockout states directly is therefore important for reducing censored-demand effects and supporting replenishment decisions.

This report consolidates the selected data-processing and exploratory analysis workflow and five model experiments into one comparative study. The first workflow performs City ID 3 filtering, validation, EDA, feature engineering, and export. The five models reuse the same modeling table to train multi-output classifiers with XGBoost, LightGBM, Random Forest, Logistic Regression, and MLP. The objective is to predict a 16-dimensional binary stockout vector for the 6:00–22:00 period (hourly slots 6–21).

Contributions. The report reconstructs the submitted data-to-model pipeline, compares five algorithms under the same target, feature set, split, and threshold-tuning procedure, and summarizes feature-importance/SHAP evidence from the five models.

2 Related Work

Fresh food stockout forecasting differs from ordinary sales forecasting because observed sales can be censored by inventory availability. FreshRetailNet-50K was introduced for stockout-annotated censored-demand research in fresh retail [1]. Its hourly stock-status arrays make it suitable for direct stockout prediction rather than indirect inference from zero sales alone.

For tabular retail data, tree ensembles are strong baselines because they capture nonlinear interactions with limited preprocessing. XGBoost applies regularized gradient-boosted trees [2]; LightGBM improves boosting efficiency through histogram-based learning and leaf-wise growth [3]; and Random Forest provides a bagging-based ensemble reference [4]. Logistic Regression remains useful as a transparent linear baseline. MLP represents a neural nonlinear baseline but may require stronger scaling and longer training for tabular retail data. Model interpretation is also essential: global feature importance identifies dominant predictors, while SHAP assigns output contributions to individual variables [5].

3 Methodology

The methodology converts the raw FreshRetailNet-50K records into a comparable multi-output stockout forecasting benchmark. To ensure a fair comparison, all five models use the same City ID 3 subset, engineered feature table, hourly target definition, chronological train–test split, threshold-tuning rule, and evaluation metrics.

Step 1. Prepare the dataset. Load the FreshRetailNet-50K training split, filter records with `city_id=3`, remove the redundant city identifier, and validate the subset for missing values and duplicate records.

Step 2. Analyze stockout patterns. Perform EDA on sales vectors, stock-status vectors, store attributes, product categories, calendar variables, and contextual variables to understand the main data characteristics before modeling.

Step 3. Construct the modeling table. Create date-derived variables, compute `oos_rate_lag1_day`, add the `is_weekend` indicator, and assemble the final 15 input features used by every model.

Step 4. Define hourly targets. Parse `hours_stock_status` and extract the 16 labels for the 6:00–22:00 period, corresponding to hourly slots 6–21.

Step 5. Apply a shared modeling protocol. Sort records by date, use the first 83 days for training and the final 7 days for testing, then wrap each base learner with `MultiOutputClassifier` so that one binary classifier is trained for each operating hour.

Step 6. Tune prediction thresholds. Convert probability outputs into binary stockout decisions using one threshold per hour, selected from the Precision–Recall curve by maximizing F1-Score.

Step 7. Evaluate and interpret results. Compute macro-averaged Accuracy, Precision, Recall, and F1-Score across the 16 outputs, then summarize feature importance and representative SHAP checks to support model interpretation.

This step-by-step design keeps preprocessing, target construction, model training, threshold tuning, and evaluation consistent across XGBoost, LightGBM, Random Forest, Logistic Regression, and MLP. Therefore, the final comparison reflects model behavior rather than differences in data preparation or evaluation settings.

4 Sequential Experimental Workflow

This section follows the workflow order: data processing, EDA, feature engineering, shared modeling protocol, and five-model comparison.

4.1 Data Processing

The data-processing workflow loads the FreshRetailNet-50K training split and filters records with `city_id=3`. After dropping the redundant city identifier, the dataframe contains 238,320 rows and 18 working columns. No missing values are found. No duplicate rows are found after excluding list-like array columns from the duplicate check. Table 1 summarizes the final subset.

Table 1. City ID 3 dataset summary after preprocessing.

Item	Value	Item	Value
Filtered records	238,320	Unique stores	101
Working columns	18	Unique products	173
Management groups	7	First-level categories	23
Second-level categories	43	Third-level categories	96
Calendar days	90	Missing values	0
Training samples	219,784	Test samples	18,536
Input features	15	Hourly targets	16

4.2 Exploratory Data Analysis

EDA confirms that the subset contains meaningful variation for stockout modeling. The all-zero `hours_sale` vector appears 26,396 times. For `hours_stock_status`, the all-zero vector appears 103,786 times, while the all-one vector appears 23,081 times. This shows that the data include both fully available and fully unavailable daily patterns. Figures 1–3 place the EDA visualizations directly after the EDA discussion and enlarge them for readability.

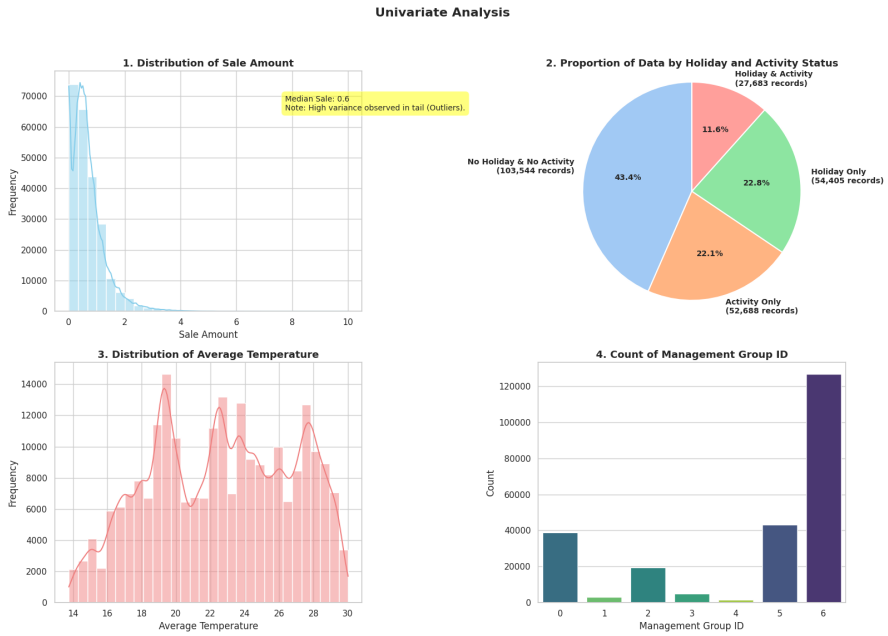

Fig. 1. Univariate EDA summary for the City ID 3 subset.

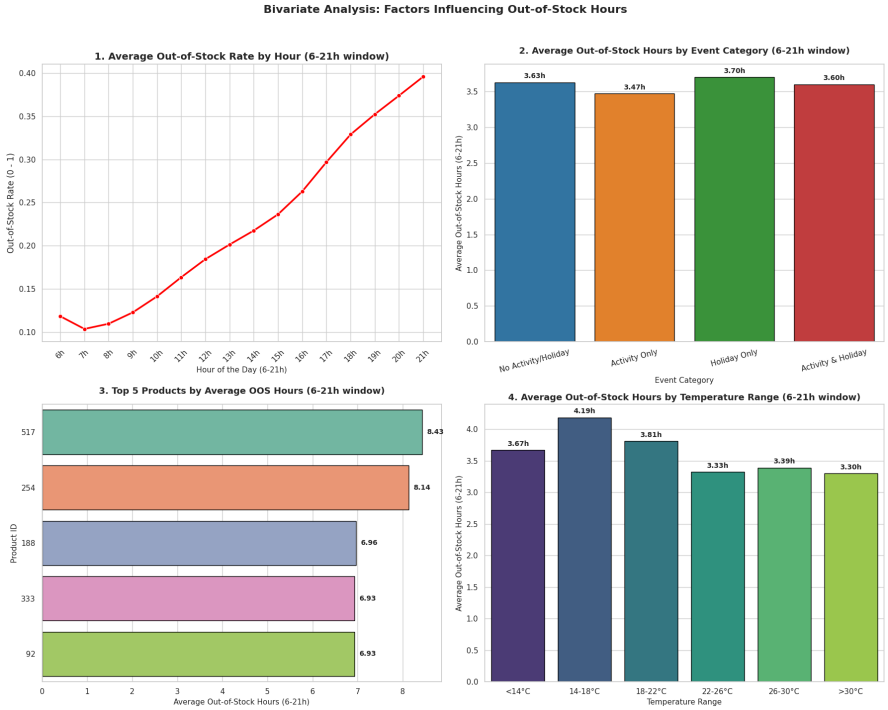

Fig. 2. Bivariate EDA summary showing relationships among key variables.

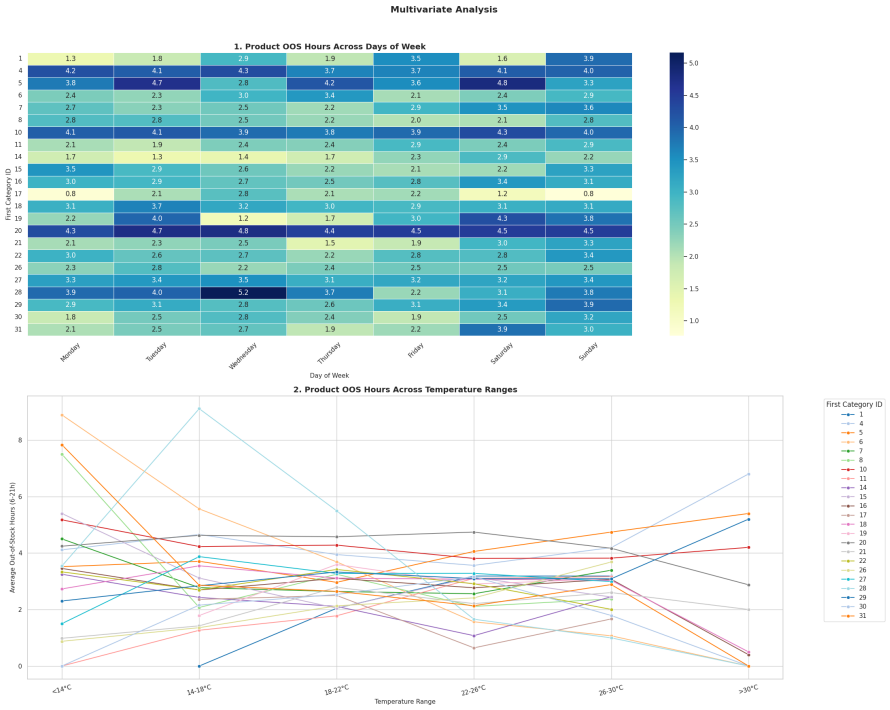

Fig. 3. Multivariate EDA summary for combined stockout, product, store, and contextual patterns.

4.3 Feature Engineering

Feature engineering creates date-derived variables and two final modeling features. The first is `oos_rate_lag1_day`, computed within each store–product pair as the previous day's stockout rate over the 6:00–22:00 window (hourly slots 6–21):

$$\texttt{oos\_rate\_lag1\_day}_{s,p,t} = \frac{\texttt{oos\_hours\_per\_record}_{s,p,t-1}}{16}. \tag{1}$$

Missing lag values are filled with 0. The second feature, `is_weekend`, produces 68,848 weekend records and 169,472 weekday records.

Each model parses `hours_stock_status`, a 24-hour stock-status vector, and slices indices 6:22 to obtain the 16 labels for hourly slots 6–21 in the 6:00–22:00 period. For store–product record i on day t, the target is

$$Y_{i,t} = [y_{i,t,6}, y_{i,t,7}, \ldots, y_{i,t,21}] \in \{0,1\}^{16}, \tag{2}$$

where $y_{i,t,h} = 1$ indicates a stockout at hour h and $y_{i,t,h} = 0$ indicates availability. Each classifier therefore produces 16 binary outputs.

All models use the same 15 features from the notebooks: store/product/category identifiers, `discount`, `holiday_flag`, `activity_flag`, weather variables, `oos_rate_lag1_day`, and `is_weekend`.

4.4 Shared Modeling Protocol

The dataset is sorted by date before splitting. The first 83 days are used for training, giving `X_train` with shape (219,784, 15); the final 7 days are used for testing, giving `X_test` with shape (18,536, 15). Chronological splitting reduces temporal leakage and better represents deployment.

Each algorithm is wrapped in `MultiOutputClassifier`, producing one binary classifier per operating hour. After training, probability outputs are converted to binary decisions with one threshold per hour. Each threshold is selected from the Precision–Recall curve by maximizing F1-Score. This is necessary because stockout frequency changes across hours, and a single cutoff of 0.5 is not appropriate.

Metrics are computed for each hour and macro-averaged across the 16 outputs:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \tag{3}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \qquad \text{F1} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{4}$$

Here, TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives for the stockout class.

5 Results

This section reports the final comparison of the five models under the shared 16-target, 15-feature, chronological-split, and hour-specific-threshold protocol.

5.1 Overall Performance

Table 2 presents the final macro-averaged test-set metrics after applying the optimized threshold for each operating hour.

Table 2. Final test-set performance after optimized hourly thresholds.

Model	Accuracy	Precision	Recall	F1-Score
XGBoost	**80.20%**	55.07%	**69.72%**	**61.36%**
LightGBM	79.93%	**55.70%**	69.00%	61.34%
Random Forest	78.80%	52.57%	67.52%	58.87%
Logistic Regression	75.72%	46.87%	59.82%	52.17%
MLP	75.58%	46.62%	60.39%	52.41%

XGBoost gives the strongest overall result, leading in Accuracy, Recall, and F1-Score. LightGBM is the closest competitor: it has the best Precision and is almost tied with XGBoost in F1-Score. Random Forest is the strongest non-boosting baseline, while Logistic Regression and MLP remain lower-performing references under the recorded setup.

5.2 Model Configuration Summary

For reproducibility, Table 3 summarizes the core settings reconstructed from the notebooks. The comparison remains fair because all five models use the same target construction, feature set, train–test split, and threshold-selection rule.

Table 3. Core model configurations.

Model	Configuration
XGBoost	`n_estimators=100`, `max_depth=7`, `learning_rate=0.05`, `eval_metric=logloss`, `random_state=42`.
LightGBM	Binary objective, `metric=binary_logloss`, `n_estimators=50`, `learning_rate=0.1`, `num_leaves=31`, `colsample_bytree=0.8`, `subsample=0.8`, L1/L2 regularization.
Random Forest	`n_estimators=25`, `max_depth=6`, `min_samples_leaf=10`, `class_weight=balanced`, `n_jobs=-1`.
Logistic Regression	`solver=liblinear`, `random_state=42`.
MLP	Hidden layers $(100, 50)$, ReLU activation, Adam solver, `max_iter=10`, early stopping, `n_iter_no_change=3`.

5.3 Threshold Behavior

Table 4 summarizes the minimum, maximum, and range of the 16 optimized hourly thresholds for each model.

Table 4. Optimized threshold ranges across 16 operating-hour targets.

Model	Min.	Max.	Range	Interpretation
XGBoost	0.257	0.367	0.110	Stable boosted-tree calibration.
LightGBM	0.254	0.386	0.132	Precision-oriented boosted-tree behavior.
Random Forest	0.402	0.722	0.320	Conservative probability scale.
Logistic Regression	0.163	0.401	0.238	Linear baseline with variable cutoffs.
MLP	0.017	0.449	0.432	Highest threshold instability.

The compact threshold ranges of XGBoost and LightGBM are consistent with their stronger F1-Score. Random Forest requires higher cutoffs, indicating a more conservative probability scale. MLP has the widest threshold range, which helps explain why it is less stable than the tree-based methods. These results support the use of hour-specific thresholds instead of a fixed 0.5 cutoff.

6 Explainability and Discussion

Feature importance is aggregated across the 16 hourly classifiers. Tree models use estimator importance, Logistic Regression uses average absolute coefficients, and MLP uses permutation importance. Figure 4 summarizes the five model-specific importance plots from the notebooks.

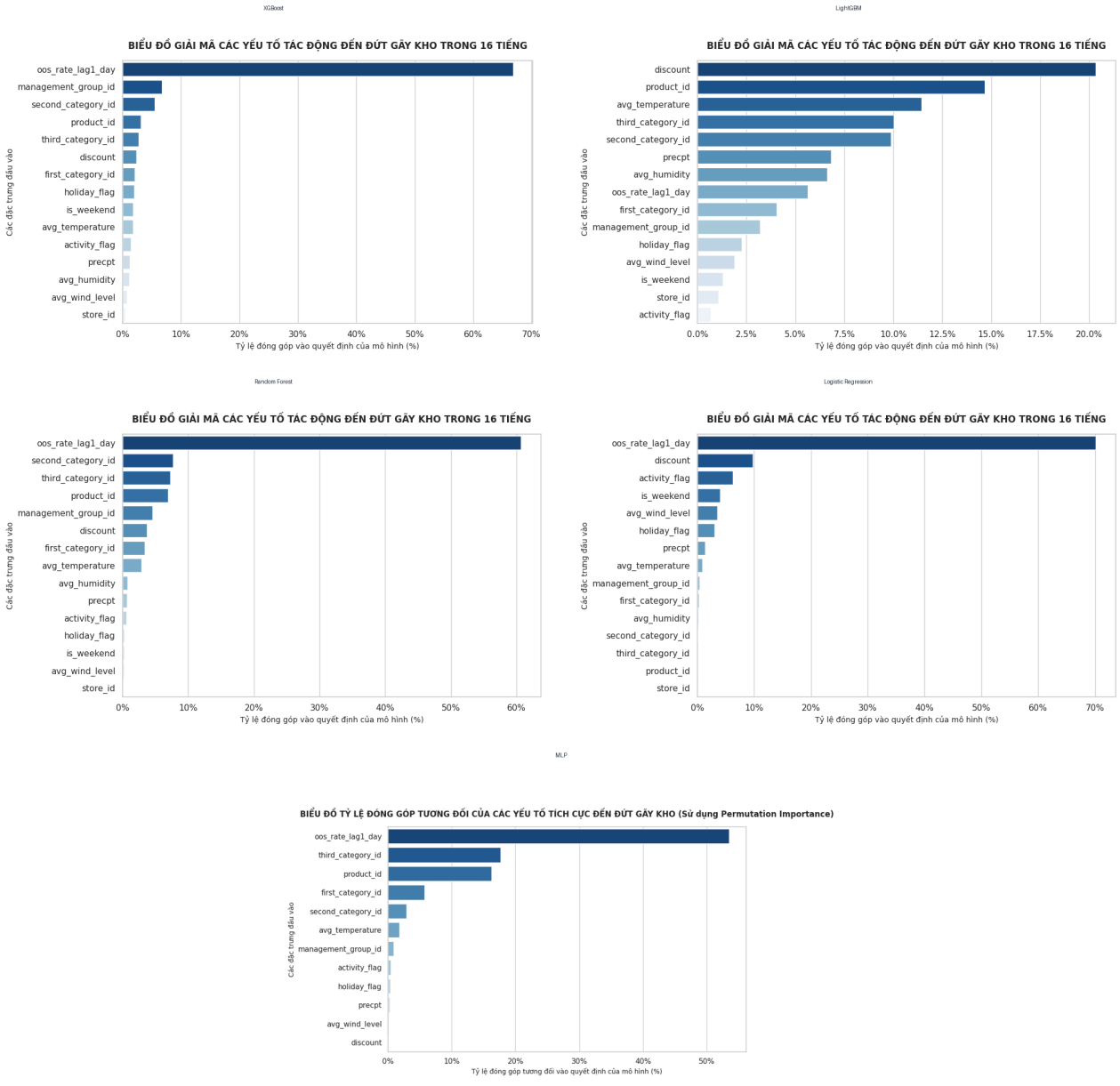

Fig. 4. Feature-importance summaries for all five models, arranged in a compact page-filling grid.

SHAP is included for the five models as a complementary check for representative hours 7, 12, and 17. The plots are not shown here to keep the report compact; the discussion relies mainly on the metrics, threshold ranges, and feature-importance summaries that are directly reported above.

For model selection, XGBoost is preferable when Recall is prioritized, while LightGBM is preferable when Precision is prioritized. The remaining models serve as baselines under the same split and thresholding protocol.

7 Conclusion

This report revisits the selected data-processing and exploratory analysis workflow and the five model experiments, condensing them into a single comparative study covering City ID 3 filtering, validation, EDA, feature engineering, target construction, five multi-output models, hour-specific threshold tuning, metrics, and explainability checks.

XGBoost is the best final model, with Accuracy of 80.20%, Recall of 69.72%, and F1-Score of 61.36%. LightGBM is a close second, with the best Precision of 55.70% and an F1-Score of 61.34%. Under the recorded feature set and split, boosted-tree methods are the strongest models in the comparative study.

References

1. Dingdong-Inc: FreshRetailNet-50K: A Stockout-Annotated Censored Demand Dataset for Latent Demand Recovery and Forecasting in Fresh Retail. arXiv:2505.16319 (2025).
2. Chen, T., Guestrin, C.: XGBoost: A Scalable Tree Boosting System. KDD, 785–794 (2016).
3. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.-Y.: LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS, 3146–3154 (2017).
4. Breiman, L.: Random Forests. Machine Learning **45**, 5–32 (2001).
5. Lundberg, S. M., Lee, S.-I.: A Unified Approach to Interpreting Model Predictions. NeurIPS, 4765–4774 (2017).